Stack

06 July 2026 18:53

Question 1: Menu-Driven Stack Implementation Using Array

Develop a **menu-driven Java application** to implement a **Stack using an array.**

The program should support the following operations:

- Push an element onto the stack.
- Pop the top element from the stack.
- Peek (display the top element).
- Display all stack elements from top to bottom.
- Check whether the stack is empty.
- Check whether the stack is full.
- Display the current size of the stack.
- Exit the program.

Additional Requirements:

- Create a separate Stack class.
- Handle **Stack Overflow** and **Stack Underflow** conditions.
- Use appropriate exception handling.
- Display user-friendly messages.

Question 2: Student Record Management Using Stack

Design and implement a **Student Record Management System** using the **Stack data structure** in Java.

Each student record should contain:

- Roll Number
- Name
- Course
- Marks

Student
(rollNum, name, course, marks)
Student(){...}
Student(int rollNum, String Name, String course, float marks){
this.rollNum
}
void setName(String name){
}
Str

The application should provide the following features:

- Add a student record (Push).
- Remove the most recently added student (Pop).
- Display the latest student record (Peek).
- Display all student records.
- Count the total number of students currently stored.
- Search for a student using the Roll Number without modifying the original stack.
- Exit the application.

Additional Requirements:

- Create a separate Student class.
- Create a custom StudentStack class.
- Use Java's object-oriented principles.
- Implement proper exception handling.

Question 3: Browser History Simulation Using Stack

Develop a **Browser History Management System** using the **Stack data structure.**

The application should simulate browser navigation with the following operations:

- Visit a new webpage.
- Display the current webpage.
- Go back to the previous webpage.
- Display complete browsing history.
- Clear all browsing history.
- Count the total visited webpages.

- Exit the application.

Additional Requirements:

- Store the following details:
  - Website URL
  - Website Title
  - Date and Time of Visit
- Create appropriate classes.
- Use stack operations to manage browser history.
- Handle situations when there is no previous page.

Question 4: Expression Processing Using Stack

Write a Java program to perform **expression conversion and evaluation** using the **Stack data structure.**

The program should allow the user to:

- Convert an **Infix Expression** to **Postfix.**
- Convert an **Infix Expression** to **Prefix.**
- Evaluate a Postfix expression.
- Evaluate a Prefix expression.
- Check whether parentheses are balanced.
- Display appropriate error messages for invalid expressions.

Additional Requirements:

- Use a stack for all conversions and evaluations.
- Implement separate methods for each operation.
- Accept expressions from the user.
- Include comments explaining the algorithm.

Question 5: Undo Functionality in a Text Editor Using Stack

Develop a **Text Editor Simulator** using the **Stack data structure** to implement the **Undo** feature.

The application should support the following operations:

- Type new text.
- Append text to the existing content.
- Delete the last entered text.
- Undo the last operation.
- Display the current document.
- Display the complete undo history.
- Clear all history.
- Exit the application.

Additional Requirements:

- Store every change in a stack.
- Implement a separate TextEditor class.
- Maintain the current document after every operation.
- Handle situations where there are no operations available to undo.
- Use appropriate exception handling and object-oriented programming concepts.